

FRACMATH: A fast and vectorized MATLAB framework for continuum damage mechanics with crack-band regularization

Jaykumar Mavani¹ and Madura Pathirage^{1*}

¹ Gerald May Department of Civil, Construction, and Environmental Engineering, University of New Mexico, Albuquerque, NM, USA * Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 22 June 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

FRACMATH is an open-source MATLAB framework for finite-element simulation of crack-band-regularized continuum damage mechanics (CDM) in quasi-brittle materials. The code uses a scalar isotropic damage variable, the modified von Mises equivalent strain (de Vree et al., 1995), exponential softening, and Oliver's direction-dependent projected crack-band length (Bažant & Oh, 1983; Oliver, 1989). It keeps the solver, plotting, and constitutive update inside MATLAB, with no MATLAB executable (MEX) files, compiled extensions, or separate build system.

The package includes a two-dimensional (2D) notched three-point bending (3PB) benchmark checked against Abaqus/Standard through an Oliver-matched user material subroutine (UMAT), plus two three-dimensional (3D) MATLAB demonstrations: the Nooru-Mohamed mixed-mode test and Brokenshire's notched beam torsion test. The repository stores source code, input decks, results, plotting scripts, and a theory manual at https://github.com/jaymavani7/JOSS_FRACMATH under the MIT license.

Statement of need

MATLAB remains useful in engineering research because students already know its matrix syntax, plotting tools, debugger, profiler, and sparse linear algebra. This matters for fracture modeling, where a user often needs to inspect element-level strain histories, change a softening law, compare crack-band lengths, and immediately visualize the damage field. FRACMATH uses that accessibility to provide a transparent CDM reference implementation rather than a general-purpose finite-element platform.

State of the field

Open-source finite-element tools already cover many research needs, including OOFEM in C++ (Patzák & Bittnar, 2001), FEniCS for Python/C++ workflows (Logg et al., 2012), Akantu for high-performance fracture simulations (Richart et al., 2024), and CALFEM as a MATLAB teaching toolbox (Austrell et al., 2004). These projects are valuable, but extending them for a new damage formulation can require learning a larger architecture, templated interface, wrapper layer, or compiled user-material workflow. Commercial tools such as Abaqus/Standard are also powerful, but custom CDM models generally require user-material subroutines, such as UMAT for Abaqus/Standard or VUMAT for Abaqus/Explicit, plus careful state-variable handling (Dassault Systèmes Simulia Corp., 2023).

The closest recent Journal of Open Source Software (JOSS) comparison is Parallel-CDM by Eldababy et al. (Eldababy et al., 2025), which provides a MATLAB implementation of 2D local and nonlocal CDM with parallel assembly. FRACMATH is complementary: its emphasis is direction-dependent Oliver crack-band scaling, the same bandwidth formula in MATLAB and Abaqus, a direct UMAT cross-check on an identical 3PB mesh, and reuse of the scalar damage routine for 3D mixed-mode and torsion-driven crack paths.

Software design

The material model follows standard scalar CDM, where damage variables and equivalent-strain histories are used to represent stiffness degradation in quasi-brittle fracture (Bažant & Planas, 1998; de Vree et al., 1995). Damage degrades the undamaged stiffness as

$$\sigma = (1 - \omega)C_0 : \varepsilon, \quad (1)$$

where $\omega \in [0, 1]$ is the damage variable. A history variable stores the maximum equivalent strain so damage cannot heal. Exponential softening is scaled element by element so the dissipated fracture energy matches G_F after crack-band regularization (Bažant & Oh, 1983). Instead of using a fixed mesh length, FRACMATH computes Oliver's projected characteristic length from the element geometry and the current maximum-principal-strain direction (Oliver, 1989). For a three-node triangular finite element (T3, a linear triangle with three corner nodes),

$$h(\mathbf{n}) = \frac{2}{\sum_{a=1}^3 |\nabla N_a \cdot \mathbf{n}|}. \quad (2)$$

The same idea is used for tetrahedra in 3D. In the Abaqus comparison, a preprocessing script writes the T3 shape-function gradients to `oliver_t3_gradN.dat`; the UMAT then recomputes Equation 2 from the current principal strain direction. This avoids using Abaqus's built-in characteristic element length variable, `CELENT`, as a fixed crack-band length and keeps the regularization consistent with Oliver's characteristic-length construction (Oliver, 1989).

In Equation 2, $h(\mathbf{n})$ is the direction-dependent projected crack-band length, $\mathbf{n}$ is the unit direction of the current maximum principal strain, N_a is the T3 shape function associated with node a , ∇N_a is its spatial gradient in the element, and the dot denotes the Euclidean inner product.

The quasistatic solver uses displacement control with a fixed-secant modified Newton iteration in each increment. The secant stiffness is assembled and factored once, displacement residuals are iterated with that fixed matrix, and damage is updated after the displacement solve. Element strains, equivalent strain, history variables, damage, and crack-band lengths are updated with vectorized MATLAB operations, including `pagetime`s; sparse linear systems use MATLAB's backslash interface, which dispatches to UMFPACK (Davis, 2004). This design favors readable vectorized kernels, direct plotting, and reviewer-side inspection over a larger compiled framework.

For the Abaqus comparison, FRACMATH includes the input deck, the Fortran UMAT, the Oliver gradient table generator, and extraction scripts for load-CMOD curves, where CMOD is crack-mouth-opening displacement, and damage fields. The MATLAB and Abaqus paths therefore share the same mesh, boundary conditions, fracture energy, tensile strength, and crack-band bandwidth formula. Differences in the plotted response are primarily solver and implementation differences, not changes in the continuum model.

78 **Benchmarks**

79 The main quantitative benchmark is the Grégoire notched 3PB beam (Grégoire et al., 2013),
80 modeled with the same refined mesh, material constants, loading, scalar damage law, and
81 Oliver crack-band formula in MATLAB and Abaqus. The 2D mesh uses Abaqus CPS3 elements,
82 where CPS3 denotes a three-node plane-stress triangular element. MATLAB predicts a peak
83 load of 3.63982 kN at CMOD 0.022811 mm; Abaqus predicts 3.60913 kN at CMOD 0.022485
84 mm. Both simulations localize damage upward from the notch, which is the expected opening-
85 mode (mode-I) crack path for this geometry (Grégoire et al., 2013). The Abaqus UMAT
86 independently evaluates the same damage law and Oliver bandwidth inside a commercial
87 finite-element environment (Dassault Systèmes Simulia Corp., 2023; Oliver, 1989).

Table 1: 2D 3PB comparison using the same mesh, material law, and Oliver T3 crack-band bandwidth.

Quantity	MATLAB	Abaqus + UMAT
Peak load (N)	3639.82	3609.13
CMOD at peak (mm)	0.022811	0.022485
Solver/submit wall-clock (s)	547.58	1996.25
End-to-end/internal time (s)	590.54	1968.00

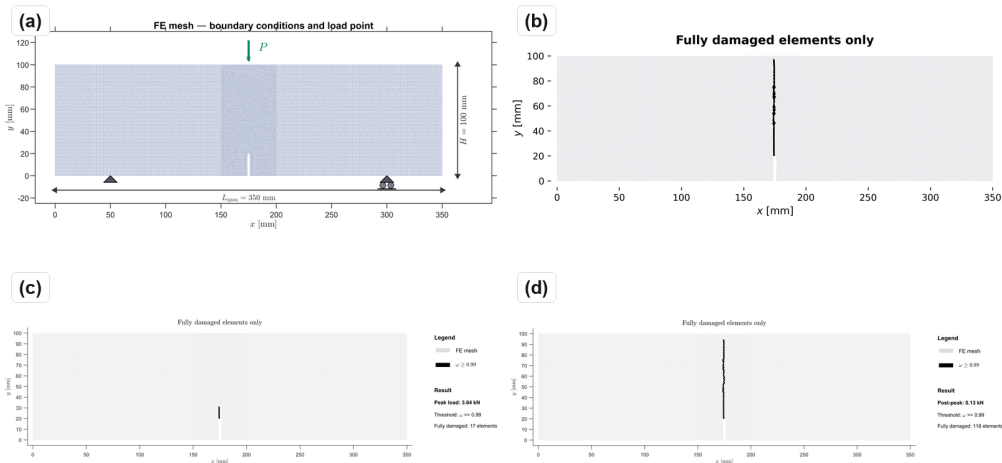


Figure 1: 3PB mesh and damage fields: (a) CPS3 mesh and support/load layout; (b) Abaqus UMAT final fully damaged elements; (c) MATLAB damage field at peak load; (d) MATLAB post-peak damage field.

88 The stored timing logs for this benchmark run report MATLAB R2024a using one thread and
89 Abaqus/Standard 2023 using four threads. The MATLAB solver wall-clock time was 547.58 s,
90 while the Abaqus submit-to-completion time was 1996.25 s. MATLAB time was dominated by
91 stiffness assembly, not by the sparse solve. The timing should not be read as a universal speed
92 claim, because solver settings, output requests, hardware, and Abaqus licensing can all change
93 wall-clock time.

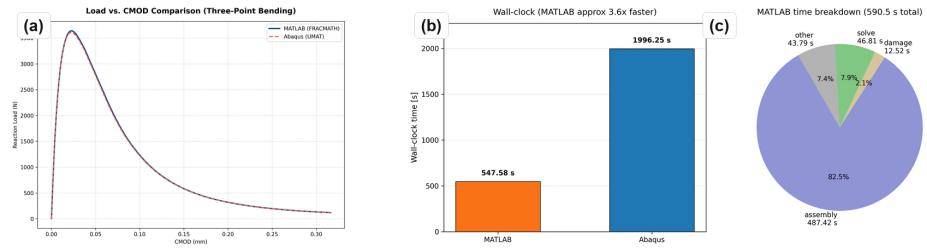


Figure 2: 3PB response and timing: (a) MATLAB and Abaqus load-CMOD response; (b) wall-clock comparison; (c) MATLAB timing breakdown.

94 The Nooru-Mohamed benchmark checks mixed-mode 3D cracking in a double-edge-notched
 95 concrete panel under combined tension and shear (Nooru-Mohamed, 1992). The same scalar
 96 CDM routine uses four-node linear tetrahedral (TET4) elements and Oliver crack-band scaling
 97 (Oliver, 1989). The simulated damage bands initiate at the two notch tips and coalesce across
 98 the ligament, matching the qualitative experimental crack-path pattern.

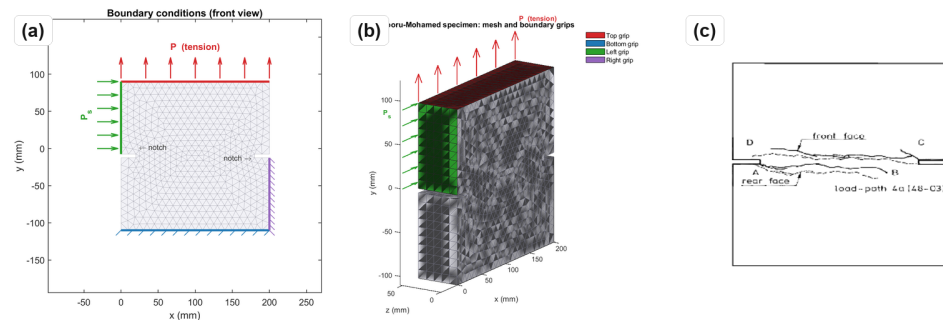


Figure 3: Nooru-Mohamed mixed-mode benchmark: (a) boundary conditions; (b) 3D mesh; (c) experimental crack path from the OOFEM gallery (OOFEM, n.d.).

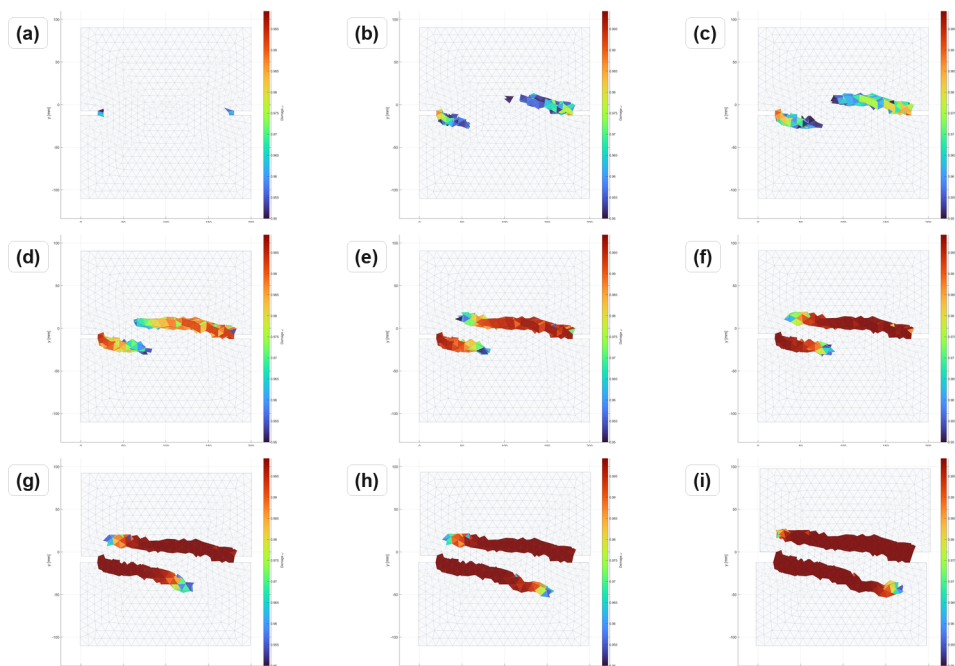


Figure 4: Nooru-Mohamed damage evolution from first localization to coalescence in a 3-by-3 sequence.

99 This example is included because mixed-mode response is a common failure point for simplified
100 fracture implementations. In the simulation, the crack-band direction changes as the principal
101 strain field evolves, so the projected bandwidth is recomputed rather than assigned from a
102 constant element size. The resulting localization band does not remain a straight mode-I
103 notch extension; it bends across the ligament in the same qualitative direction as the reported
104 experimental crack path.

105 Brokenshire's torsion benchmark tests whether the same formulation can recover a curved 3D
106 fracture surface in a notched plain concrete beam (Jefferson et al., 2004). The model uses a
107 prescribed twist, TET4 elements, and the same damage update. The computed band nucleates
108 at the notch front and rotates toward the loaded corner, consistent with the experimentally
109 recovered fracture surface.

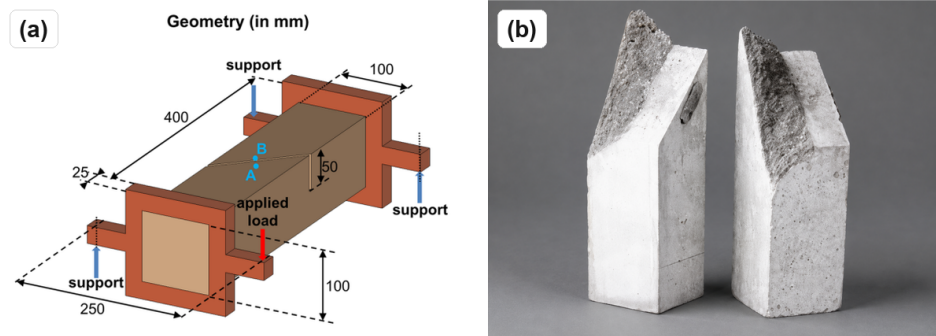


Figure 5: Brokenshire torsion benchmark: geometry and experimental fractured specimen from Jefferson et al. (Jefferson et al., 2004).

110 The torsion case is deliberately different from the 3PB validation: it contains out-of-plane
111 cracking, a nonuniform stress state, and a visibly curved fracture surface. The 3D examples
112 are intended as qualitative crack-path demonstrations; only the 2D 3PB case is quantitatively
113 cross-checked against Abaqus.

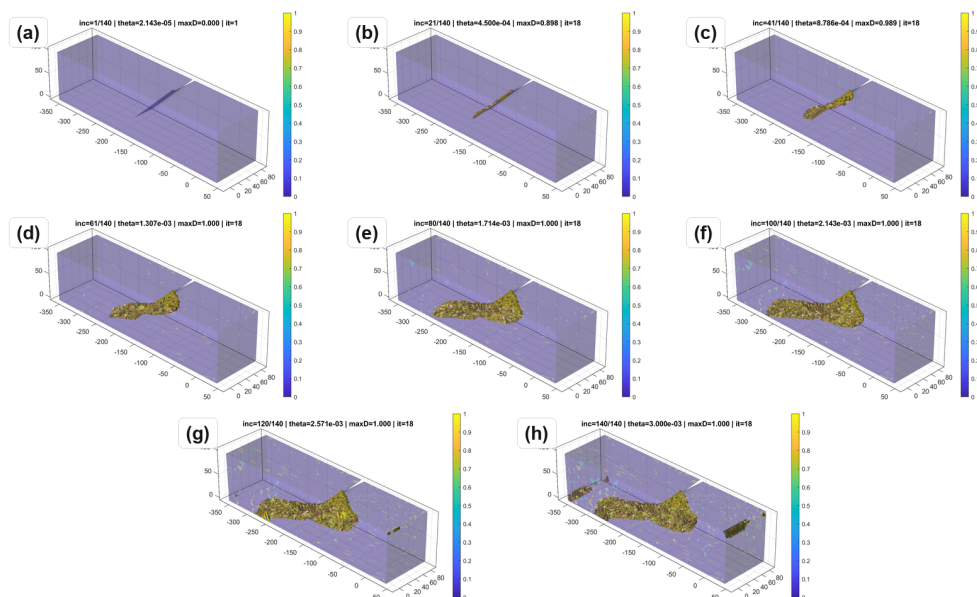


Figure 6: Torsion damage evolution over the imposed twist history.

114 Research impact statement

115 FRACMATH provides a reproducible reference workflow for CDM and crack-band studies. Its
116 immediate scholarly value is the paired MATLAB/Abaqus 3PB benchmark, where the MATLAB
117 solver is cross-checked against an independent Abaqus UMAT on the same mesh and material
118 data. The repository also provides benchmark inputs, stored output data, plotting scripts,
119 and 2D and 3D examples that can be reused for teaching, method comparison, and future
120 extensions of crack-band regularization.

121 Software availability

122 The repository contains MATLAB source code, Abaqus UMAT files, benchmark input decks,
123 plotting scripts, updated results, a theory manual, LICENSE, CITATION.cff, CONTRIBUTING.md,
124 and reproducibility instructions. Reviewers can run the smoke tests from the repository root with
125 `matlab -batch "addpath('tests'); run_smoke_checks"`. The test suite has been checked
126 on MATLAB R2022a, R2023a, and R2024a across Linux, macOS, and Windows. A release
127 archive digital object identifier (DOI) should be added before final JOSS acceptance.

128 Limitations

129 FRACMATH is quasistatic and does not include inertia, rate effects, contact, plasticity, or unilateral
130 crack closure. Damage is isotropic and scalar, so anisotropic stiffness recovery and cyclic loading
131 are outside the current scope. Crack-band scaling controls mesh-objective energy dissipation
132 but does not introduce a physical material length scale. These additions are compatible with
133 the current framework, because the code already separates element-level history variables,
134 constitutive updates, sparse assembly, and plotting.

135 AI usage disclosure

136 Generative AI tools were used to assist with manuscript wording and formatting. The scientific
137 claims, equations, code, numerical results, figures, and references are the responsibility of the
138 authors.

139 Acknowledgements

140 The authors acknowledge support from the National Aeronautics and Space Administration
141 (NASA), and the New Mexico Space Grant Consortium under grant 80NSSC22M0044, and from
142 the New Mexico Department of Finance and Administration under grant ZI5044-MG25-109.
143 The opinions and conclusions are those of the authors and do not necessarily reflect the views
144 of the sponsors.

145 References

- 146 Austrell, P.-E., Dahlblom, O., Lindemann, J., Olsson, A., Olsson, K.-G., Persson, K., Petersson,
147 H., Ristinmaa, M., Sandberg, G., & Wernberg, P.-A. (2004). *CALFEM — a finite element*
148 *toolbox, version 3.4*. Lund University.
- 149 Bažant, Z. P., & Oh, B. H. (1983). Crack band theory for fracture of concrete. *Matériaux Et*
150 *Construction*, 16(3), 155–177. <https://doi.org/10.1007/BF02486267>
- 151 Bažant, Z. P., & Planas, J. (1998). *Fracture and size effect in concrete and other quasibrittle*
152 *materials*. CRC Press.

- 153 Dassault Systèmes Simulia Corp. (2023). *Abaqus/standard user's manual, version 2023*.
- 154 Davis, T. A. (2004). Algorithm 832: UMFPACK V4.3 — an unsymmetric-pattern multifrontal
155 method. *ACM Transactions on Mathematical Software*, 30(2), 196–199. <https://doi.org/10.1145/992200.992206>
156
- 157 de Vree, J. H. P., Brekelmans, W. A. M., & van Gils, M. A. J. (1995). Comparison of nonlocal
158 approaches in continuum damage mechanics. *Computers & Structures*, 55(4), 581–588.
159 [https://doi.org/10.1016/0045-7949\(94\)00501-S](https://doi.org/10.1016/0045-7949(94)00501-S)
- 160 Eldababy, H., Saji, R. P., Pantidis, P., & Mobasher, M. (2025). Parallel-CDM: Parallel
161 implementation of continuum damage mechanics simulations using FEM and MATLAB.
162 *Journal of Open Source Software*, 10(112), 7610. <https://doi.org/10.21105/joss.07610>
- 163 Grégoire, D., Rojas-Solano, L. B., & Pijaudier-Cabot, G. (2013). Failure and size effect for
164 notched and unnotched concrete beams. *International Journal for Numerical and Analytical*
165 *Methods in Geomechanics*, 37(10), 1434–1452. <https://doi.org/10.1002/nag.2180>
- 166 Jefferson, A. D., Bennett, T., & Hee, S. C. (2004). Three-dimensional finite element simulations
167 of fracture tests using the Craft concrete model. *Computers and Concrete*, 1(3), 261–284.
168 <https://doi.org/10.12989/cac.2004.1.3.261>
- 169 Logg, A., Mardal, K.-A., & Wells, G. N. (Eds.). (2012). *Automated solution of differential*
170 *equations by the finite element method: The FEniCS book*. Springer. <https://doi.org/10.1007/978-3-642-23099-8>
171
- 172 Nooru-Mohamed, M. B. (1992). *Mixed-mode fracture of concrete: An experimental approach*
173 [PhD thesis, Delft University of Technology]. [https://resolver.tudelft.nl/uuid:a6a773f1-](https://resolver.tudelft.nl/uuid:a6a773f1-dacd-4598-aa6a-960dddf71117)
174 [dacd-4598-aa6a-960dddf71117](https://resolver.tudelft.nl/uuid:a6a773f1-dacd-4598-aa6a-960dddf71117)
- 175 Oliver, J. (1989). A consistent characteristic length for smeared cracking models. *International*
176 *Journal for Numerical Methods in Engineering*, 28(2), 461–474. [https://doi.org/10.1002/](https://doi.org/10.1002/nme.1620280214)
177 [nme.1620280214](https://doi.org/10.1002/nme.1620280214)
- 178 OOFEM. (n.d.). *Adaptive simulation of nooru-mohamed test*. Retrieved June 22, 2026, from
179 https://ksm.fsv.cvut.cz/oofem/gallery/nooru-mohamed/Nooru-Mohamed_test.html
- 180 Patzák, B., & Bittnar, Z. (2001). Design of object-oriented finite element code. *Advances*
181 *in Engineering Software*, 32(10–11), 759–767. [https://doi.org/10.1016/S0965-9978\(01\)](https://doi.org/10.1016/S0965-9978(01)00027-8)
182 [00027-8](https://doi.org/10.1016/S0965-9978(01)00027-8)
- 183 Richart, N., Anciaux, G., Gallyamov, E., Frérot, L., Kammer, D., Pundir, M., Vocialta,
184 M., Ramos, A. C., Corrado, M., Müller, P., Barras, F., Zhang, S., Ferry, R., Durussel,
185 V., & Molinari, J.-F. (2024). Akantu: An HPC finite-element library for contact and
186 dynamic fracture simulations. *Journal of Open Source Software*, 9(94), 5253. <https://doi.org/10.21105/joss.05253>
187